

Atualizacao remota de firmware (OTA) em ESP32

Solucao simulada no Wokwi para um no IoT de monitoramento da altura da vegetacao, com medicao periodica, versionamento, download HTTPS e evolucao funcional do firmware.

Disciplina	Sistemas embarcados / Projeto Motiva
Professor	Marcelo Fernando Morgantini
Sala	CCPX
Entrega	Setembro de 2026

Links publicos

Projeto Wokwi: <https://wokwi.com/projects/475874796669478913>

Repositorio: <https://github.com/vitorlimeira22/motiva-ota-esp32-cp2>

1. Identificacao da equipe

Integrante	RM
Vitor Limeira dos Santos	565280
Pedro Del Neri Correia	562168
Lucas de Freitas Barbosa	564685
Arthur da Silva Alecar	563684
Felipe Paula Burba Molonhoni	564395

2. Resumo executivo

O projeto implementa duas versoes de firmware para um ESP32 simulado. A versao 1.0 gera cinco leituras pseudoaleatorias entre 10 e 20 cm, calcula a media e inicia cada sessao exatamente a cada 48 segundos. Depois de tres sessoes, conecta-se a rede Wokwi-GUEST, consulta um manifesto remoto, compara versoes, baixa o binario 2.0 e grava a atualizacao na particao OTA.

Depois do reinicio, a versao 2.0 mantem as funcoes anteriores e acrescenta ordenacao, mediana e histerese. O LED RGB identifica tanto a versao instalada quanto o estado NORMAL ou ALERTA.

RESULTADO DE ENGENHARIA: os dois firmwares foram compilados com Arduino-ESP32 3.3.12. O binario da versao 2.0 possui 274.320 bytes e os seis testes automatizados do projeto foram aprovados.

3. Arquitetura da solucao



Fluxo OTA: 3 sessoes -> Wi-Fi -> comparar versao -> baixar -> gravar -> reiniciar

O manifesto **version.json** e o arquivo **firmware_v2.bin** permanecem no repositorio publico. O Firmware 1.0 usa HTTPS, acompanha redirecionamentos e valida o codigo HTTP e o tamanho do arquivo antes de iniciar a gravacao.

Componente	Responsabilidade
ESP32 / Firmware 1.0	Medicoes, media, LED azul, tres sessoes e cliente OTA.
Wokwi-GUEST	Acesso simulado a Internet no canal 6.
version.json	Informa a versao 2.0 e a URL direta do binario.
firmware_v2.bin	Imagem compilada gravada na particao OTA inativa.
ESP32 / Firmware 2.0	Media, ordenacao, mediana, histerese e LED verde/vermelho.

4. Firmware 1.0

A primeira versao identifica-se no Serial Monitor, mantem o LED azul e armazena cinco alturas em um vetor. O gerador congruente linear utiliza a semente 87 para tornar a demonstracao reproduzivel sem deixar de produzir valores pseudoaleatorios na faixa solicitada.

4.1 Temporizacao sem deriva



A agenda usa **millis()**. O instante da proxima sessao e incrementado em 48.000 ms a partir do inicio programado, e nao depois da quinta leitura. Assim, a sessao seguinte ocorre em 48 s, nao em 56 s.

Requisito	Implementacao
5 leituras	Vetor fixo com cinco inteiros entre 10 e 20 cm.
Intervalo de 2 s	proximaLeituraEm e comparacao segura de millis().
Media	Soma em long e divisao por 5.0.
Sessao a cada 48 s	proximaSessaoEm += 48000, sem delay longo.
LED da versao	RGB em azul pelos GPIOs 25, 26 e 27.

5. Firmware 2.0

A nova versao preserva a aquisicao e a media. Uma copia do vetor e ordenada pelo metodo bubble sort; o terceiro elemento ordenado e adotado como mediana. O vetor original permanece intacto para que as duas ordens sejam apresentadas no Serial Monitor.

Condicao da mediana	Decisao	LED
≥ 16 cm	Entrar em ALERTA	Vermelho
> 14 cm e < 16 cm	Manter o estado anterior	Mantem a cor do estado
≤ 14 cm	Entrar ou retornar a NORMAL	Verde

A zona intermediaria impede alternancias rapidas quando a medida se aproxima do limiar. Nas primeiras tres sessoes reproduziveis, as medianas sao 18, 15 e 14 cm. A sequencia resultante e ALERTA, ALERTA mantido e NORMAL.

6. Processo de atualizacao OTA

Etapa	Acao e validacao
1	Executar tres sessoes completas no Firmware 1.0.
2	Conectar a Wokwi-GUEST no canal 6, com limite de espera de 10 s.
3	Consultar o manifesto publico: https://raw.githubusercontent.com/vitorlmeira22/motiva-ota-esp32-cp2/main/version.json
4	Extrair version e url e comparar os componentes numericos da versao.
5	Baixar firmware_v2.bin por HTTPS e exigir resposta HTTP 200.
6	Validar Content-Length e disponibilidade de espaco para a particao OTA.
7	Gravar o fluxo recebido com Update.writeStream e conferir bytes gravados.
8	Finalizar, validar a imagem, chamar ESP.restart() e iniciar o Firmware 2.0.

6.1 Tratamento minimo de situacoes

Situacao	Mensagem / resposta do programa
Sem Wi-Fi	[OTA][ERRO] Nao foi possivel conectar ao Wi-Fi.
Manifesto inacessivel	Informa o codigo HTTP ou falha ao preparar a consulta.
Versao ja atual	A versao instalada ja e a mais recente.
Binario inacessivel	Informa o codigo HTTP ou tamanho de arquivo invalido.
Falha de gravacao	Informa bytes gravados e codigo retornado por Update.

7. Testes e evidencias

#	Condicao	Resultado verificado	Status
1	Firmware 1.0	5 leituras, media e LED azul.	APROVADO
2	Sessao completa	0, 2, 4, 6, 8 s; nova sessao em 48 s.	APROVADO
3	Manifesto 2.0	Consulta disparada somente depois de 3 sessoes.	APROVADO
4	OTA	Download, validacao, gravacao e reinicio implementados.	COMPILADO
5	Firmware 2.0	Media, original, crescente e mediana.	APROVADO
6	Mediana 18	ALERTA e LED vermelho.	APROVADO
7	Mediana 15	Estado ALERTA mantido.	APROVADO
8	Mediana 14	NORMAL e LED verde.	APROVADO

7.1 Sequencia reproduzivel da demonstracao

Sessao	Leituras	Media	Mediana	Estado
FW 2.0 / 1	18 19 19 10 10	15,2	18	ALERTA
FW 2.0 / 2	18 15 13 18 13	15,4	15	ALERTA mantido
FW 2.0 / 3	12 14 13 17 15	14,2	14	NORMAL

O teste automatizado tambem valida 1.000 valores do gerador, garantindo que todos permanecem entre 10 e 20 cm, e confirma os tres ramos da histerese.

8. Instrucoes de execucao

Passo	Instrucao
1	Abrir o projeto publico no Wokwi: https://wokwi.com/projects/475874796669478913
2	Iniciar a simulacao e manter o Serial Monitor visivel em 115200 baud.
3	Observar tres sessoes do Firmware 1.0; o LED permanece azul.
4	Aguardar a conexao, a consulta do manifesto, o download e o reinicio.
5	Confirmar o cabecalho FW 2.0 e a exibicao de media, ordem, mediana e estado.
6	Observar o LED vermelho, a manutencao do estado e o retorno ao verde.

9. Rastreabilidade dos artefatos

Artefato	Finalidade / verificacao
sketch.ino	Codigo carregado no projeto Wokwi; corresponde ao Firmware 1.0.
firmware_v1.ino	Fonte completo da versao inicial e cliente OTA.
firmware_v2.ino	Fonte completo da versao com mediana e histerese.
version.json	Manifesto remoto apontando para firmware_v2.bin.
firmware_v2.bin	274.320 bytes; SHA-256 1FF5FDA08071BAE3...1AE34D98330C5372.
diagram.json	ESP32, LED RGB catodo comum e resistores de 220 ohms.
test_logic.py	Seis testes automatizados de faixa, agenda, manifesto e histerese.

10. Conclusao

A solucao atende ao objetivo de demonstrar a manutencao remota de um equipamento instalado em campo. A versao 1.0 coleta e resume as medicoes, enquanto a versao 2.0 acrescenta um tratamento mais robusto e uma decisao estavel por histerese. O processo OTA reduz a necessidade de acesso fisico ao dispositivo, permite corrigir ou evoluir o software e preserva a rastreabilidade por versao e manifesto.

Os fontes, o binario, o manifesto, o circuito, os testes e as instrucoes estao centralizados no repositorio publico e devem permanecer disponiveis por pelo menos dez dias para validacao.

11. Referencias tecnicas

Fonte	URL
Wokwi - ESP32 Wi-Fi Networking	https://docs.wokwi.com/guides/esp32-wifi
Wokwi - RGB LED Reference	https://docs.wokwi.com/parts/wokwi-rgb-led
Espressif - ESP32 OTA	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html

Apendice A -Codigo completo do Firmware 1.0

Arquivo publicado como **firmware_v1.ino** e utilizado como **sketch.ino** no Wokwi

```

1  #include <WiFi.h>
2  #include <WiFiClientSecure.h>
3  #include <HTTPClient.h>
4  #include <Update.h>
5
6  const char *VERSAO_ATUAL = "1.0";
7  const char *SSID = "Wokwi-GUEST";
8  const char *URL_MANIFESTO = "https://raw.githubusercontent.com/vitorlimeira22/motiva-ota-esp32-cp2/main/version.json";
9
10 const uint8_t PINO_LED_VERMELHO = 25;
11 const uint8_t PINO_LED_VERDE = 26;
12 const uint8_t PINO_LED_AZUL = 27;
13
14 const unsigned long INTERVALO_LEITURA_MS = 2000;
15 const unsigned long INTERVALO_SESSAO_MS = 48000;
16 const uint8_t TOTAL_LEITURAS = 5;
17 const uint8_t SESSOES_ANTES_OTA = 3;
18
19 int leituras[TOTAL_LEITURAS];
20 uint8_t indiceLeitura = 0;
21 uint8_t sessoesConcluidas = 0;
22 unsigned long proximaLeituraEm = 0;
23 unsigned long proximaSessaoEm = 0;
24 bool sessaoAtiva = false;
25 bool verificarOta = false;
26
27 // Semente fixa para uma demonstracao reproduzivel. Os valores continuam
28 // sendo gerados por um algoritmo pseudoaleatorio.
29 uint32_t estadoPseudoaleatorio = 87;
30
31 void definirLed(bool vermelho, bool verde, bool azul) {
32   digitalWrite(PINO_LED_VERMELHO, vermelho ? HIGH : LOW);
33   digitalWrite(PINO_LED_VERDE, verde ? HIGH : LOW);
34   digitalWrite(PINO_LED_AZUL, azul ? HIGH : LOW);
35 }
36
37 int gerarAltura() {
38   estadoPseudoaleatorio = 1664525UL * estadoPseudoaleatorio + 1013904223UL;
39   return 10 + (estadoPseudoaleatorio % 11);
40 }
41
42 bool instanteAtingido(unsigned long agora, unsigned long alvo) {
43   return static_cast<long>(agora - alvo) >= 0;
44 }
45
46 void iniciarSessao(unsigned long inicioProgramado) {
47   indiceLeitura = 0;
48   sessaoAtiva = true;
49   proximaLeituraEm = inicioProgramado;
50
51   Serial.println();
52   Serial.println("-----");
53   Serial.printf("INICIO DA SESSAO %u - FW %s\n", sessoesConcluidas + 1, VERSAO_ATUAL);
54   Serial.println("-----");
55 }
56
57 void concluirSessao() {
58   long soma = 0;
59   for (uint8_t i = 0; i < TOTAL_LEITURAS; i++) {
60     soma += leituras[i];
61   }
62
63   float media = soma / static_cast<float>(TOTAL_LEITURAS);
64   sessoesConcluidas++;
65   sessaoAtiva = false;
66
67   Serial.printf("Media da sessao: %.1f cm\n", media);
68   Serial.printf("Sessao %u concluida.\n", sessoesConcluidas);
69   Serial.println("Proxima sessao no marco de 48 segundos.");
70
71   if (sessoesConcluidas >= SESSOES_ANTES_OTA) {
72     verificarOta = true;
73   }
74 }
75
76 void processarLeituras(unsigned long agora) {
77   if (!sessaoAtiva || indiceLeitura >= TOTAL_LEITURAS ||
78       !instanteAtingido(agora, proximaLeituraEm)) {
79     return;
80   }
81
82   leituras[indiceLeitura] = gerarAltura();

```

Continua - bloco 2 de 4

```

83   Serial.printf("Leitura %u: %d cm\n", indiceLeitura + 1, leituras[indiceLeitura]);
84   indiceLeitura++;
85   proximaLeituraEm += INTERVALO_LEITURA_MS;
86
87   if (indiceLeitura == TOTAL_LEITURAS) {
88       concluirSessao();
89   }
90 }
91
92 bool conectarWifi() {
93     if (WiFi.status() == WL_CONNECTED) {
94         return true;
95     }
96
97     Serial.println();
98     Serial.println("[OTA] Conectando a rede Wokwi-GUEST...");
99     WiFi.mode(WIFI_STA);
100    WiFi.begin(SSID, "", 6);
101
102    unsigned long limite = millis() + 10000;
103    while (WiFi.status() != WL_CONNECTED && !instanteAtingido(millis(), limite)) {
104        delay(100);
105        Serial.print('.');
106    }
107    Serial.println();
108
109    if (WiFi.status() != WL_CONNECTED) {
110        Serial.println("[OTA][ERRO] Nao foi possivel conectar ao Wi-Fi.");
111        return false;
112    }
113
114    Serial.print("[OTA] Wi-Fi conectado. IP: ");
115    Serial.println(WiFi.localIP());
116    return true;
117 }
118
119 bool extrairCampoJson(const String &json, const String &campo, String &valor) {
120     String chave = "\"" + campo + "\"";
121     int inicioChave = json.indexOf(chave);
122     if (inicioChave < 0) {
123         return false;
124     }
125
126     int doisPontos = json.indexOf(':', inicioChave + chave.length());
127     int primeiraAspa = json.indexOf('"', doisPontos + 1);
128     int segundaAspa = json.indexOf('"', primeiraAspa + 1);
129     if (doisPontos < 0 || primeiraAspa < 0 || segundaAspa < 0) {
130         return false;
131     }
132
133     valor = json.substring(primeiraAspa + 1, segundaAspa);
134     valor.trim();
135     return valor.length() > 0;
136 }
137
138 int compararVersoes(const String &a, const String &b) {
139     int posicaoA = 0;
140     int posicaoB = 0;
141
142     while (posicaoA < a.length() || posicaoB < b.length()) {
143         int fimA = a.indexOf('.', posicaoA);
144         int fimB = b.indexOf('.', posicaoB);
145         if (fimA < 0) fimA = a.length();
146         if (fimB < 0) fimB = b.length();
147
148         int parteA = a.substring(posicaoA, fimA).toInt();
149         int parteB = b.substring(posicaoB, fimB).toInt();
150         if (parteA != parteB) {
151             return parteA > parteB ? 1 : -1;
152         }
153
154         posicaoA = fimA + 1;
155         posicaoB = fimB + 1;
156     }
157     return 0;
158 }
159
160 bool baixarEInstalarFirmware(const String &urlFirmware) {
161     WiFiClientSecure clienteSeguro;
162     clienteSeguro.setInsecure();
163
164     HTTPClient http;

```

Continua - bloco 3 de 4

```

165 http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS);
166 if (!http.begin(clienteSeguro, urlFirmware)) {
167     Serial.println("[OTA][ERRO] Nao foi possivel iniciar o download do firmware.");
168     return false;
169 }
170
171 Serial.println("[OTA] Baixando firmware_v2.bin...");
172 int codigoHttp = http.GET();
173 if (codigoHttp != HTTP_CODE_OK) {
174     Serial.printf("[OTA][ERRO] Download retornou HTTP %d.\n", codigoHttp);
175     http.end();
176     return false;
177 }
178
179 int tamanho = http.getSize();
180 if (tamanho <= 0) {
181     Serial.println("[OTA][ERRO] O servidor nao informou um tamanho de firmware valido.");
182     http.end();
183     return false;
184 }
185
186 if (!Update.begin(tamanho)) {
187     Serial.printf("[OTA][ERRO] Espaco insuficiente.Codigo %u.\n", Update.getError());
188     http.end();
189     return false;
190 }
191
192 size_t gravados = Update.writeStream(*http.getStreamPtr());
193 if (gravados != static_cast<size_t>(tamanho)) {
194     Serial.printf("[OTA][ERRO] Foram gravados %u de %d bytes.\n",
195         static_cast<unsigned int>(gravados), tamanho);
196     Update.abort();
197     http.end();
198     return false;
199 }
200
201 if (!Update.end() || !Update.isFinished()) {
202     Serial.printf("[OTA][ERRO] Falha ao finalizar a atualizacao. Codigo %u.\n",
203         Update.getError());
204     http.end();
205     return false;
206 }
207
208 http.end();
209 Serial.println("[OTA] Firmware 2.0 gravado com sucesso.");
210 Serial.println("[OTA] Reiniciando o ESP32...");
211 delay(1000);
212 ESP.restart();
213 return true;
214 }
215
216 void consultarAtualizacao() {
217     verificarOta = false;
218
219     Serial.println();
220     Serial.printf("[OTA] Tres sessoes concluidas. Versao instalada: %s\n", VERSAO_ATUAL);
221     if (!conectarWifi()) {
222         return;
223     }
224
225     WiFiClientSecure clienteSeguro;
226     clienteSeguro.setInsecure();
227     HTTPClient http;
228     http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS);
229
230     if (!http.begin(clienteSeguro, URL_MANIFESTO)) {
231         Serial.println("[OTA][ERRO] Nao foi possivel preparar a consulta ao manifesto.");
232         return;
233     }
234
235     Serial.println("[OTA] Consultando version.json...");
236     int codigoHttp = http.GET();
237     if (codigoHttp != HTTP_CODE_OK) {
238         Serial.printf("[OTA][ERRO] Manifesto indisponivel. HTTP %d.\n", codigoHttp);
239         http.end();
240         return;
241     }
242
243     String manifesto = http.getString();
244     http.end();
245
246     String versaoDisponivel;

```

Continua - bloco 4 de 4

```

247 String urlFirmware;
248 if (!extrairCampoJson(manifesto, "version", versaoDisponivel) ||
249     !extrairCampoJson(manifesto, "url", urlFirmware)) {
250     Serial.println("[OTA][ERRO] Manifesto invalido: campos version e url sao obrigatorios.");
251     return;
252 }
253
254 Serial.printf("[OTA] Versao disponivel: %s\n", versaoDisponivel.c_str());
255 if (compararVersoes(versaoDisponivel, VERSAO_ATUAL) <= 0) {
256     Serial.println("[OTA] A versao instalada ja e a mais recente.");
257     return;
258 }
259
260 Serial.println("[OTA] Nova versao encontrada. Iniciando atualizacao.");
261 baixarEInstalarFirmware(urlFirmware);
262 }
263
264 void setup() {
265     Serial.begin(115200);
266     pinMode(PINO_LED_VERMELHO, OUTPUT);
267     pinMode(PINO_LED_VERDE, OUTPUT);
268     pinMode(PINO_LED_AZUL, OUTPUT);
269     definirLed(false, false, true);
270
271     Serial.println();
272     Serial.println("=====");
273     Serial.println("MONITORAMENTO DE VEGETACAO - FW 1.0");
274     Serial.println("LED AZUL: FIRMWARE 1.0 EM EXECUCAO");
275     Serial.println("=====");
276
277     unsigned long agora = millis();
278     iniciarSessao(agora);
279     proximaSessaoEm = agora + INTERVALO_SESSAO_MS;
280 }
281
282 void loop() {
283     unsigned long agora = millis();
284     processarLeituras(agora);
285
286     if (!sessaoAtiva && instanteAtingido(agora, proximaSessaoEm)) {
287         unsigned long inicioProgramado = proximaSessaoEm;
288         proximaSessaoEm += INTERVALO_SESSAO_MS;
289         iniciarSessao(inicioProgramado);
290     }
291
292     if (verificarOta && !sessaoAtiva) {
293         consultarAtualizacao();
294     }
295
296     delay(10);
297 }

```

Apendice B - Codigo completo do Firmware 2.0

Fonte que gerou o arquivo binario usado na atualizacao OTA

```

1  #include <Arduino.h>
2
3  const char *VERSAO_ATUAL = "2.0";
4
5  const uint8_t PINO_LED_VERMELHO = 25;
6  const uint8_t PINO_LED_VERDE = 26;
7  const uint8_t PINO_LED_AZUL = 27;
8
9  const unsigned long INTERVALO_LEITURA_MS = 2000;
10 const unsigned long INTERVALO_SESSAO_MS = 48000;
11 const uint8_t TOTAL_LEITURAS = 5;
12
13 enum EstadoVegetacao {
14     NORMAL,
15     ALERTA
16 };
17
18 int leituras[TOTAL_LEITURAS];
19 uint8_t indiceLeitura = 0;
20 uint16_t sessoesConcluidas = 0;
21 unsigned long proximaLeituraEm = 0;
22 unsigned long proximaSessaoEm = 0;
23 bool sessaoAtiva = false;
24 EstadoVegetacao estadoAtual = NORMAL;
25
26 // Semente fixa para que os tres primeiros ciclos comprovem os tres ramos
27 // da histerese: ALERTA, manutencao do estado e NORMAL.
28 uint32_t estadoPseudoaleatorio = 87;
29
30 void definirLed(bool vermelho, bool verde, bool azul) {
31     digitalWrite(PINO_LED_VERMELHO, vermelho ? HIGH : LOW);
32     digitalWrite(PINO_LED_VERDE, verde ? HIGH : LOW);
33     digitalWrite(PINO_LED_AZUL, azul ? HIGH : LOW);
34 }
35
36 void atualizarLedDoEstado() {
37     if (estadoAtual == ALERTA) {
38         definirLed(true, false, false);
39     } else {
40         definirLed(false, true, false);
41     }
42 }
43
44 const char *nomeDoEstado(EstadoVegetacao estado) {
45     return estado == ALERTA ? "ALERTA" : "NORMAL";
46 }
47
48 int gerarAltura() {
49     estadoPseudoaleatorio = 1664525UL * estadoPseudoaleatorio + 1013904223UL;
50     return 10 + (estadoPseudoaleatorio % 11);
51 }
52
53 bool instanteAtingido(unsigned long agora, unsigned long alvo) {
54     return static_cast<long>(agora - alvo) >= 0;
55 }
56
57 void ordenarCopia(const int origem[], int destino[], uint8_t tamanho) {
58     for (uint8_t i = 0; i < tamanho; i++) {
59         destino[i] = origem[i];
60     }
61
62     for (uint8_t i = 0; i < tamanho - 1; i++) {
63         for (uint8_t j = 0; j < tamanho - i - 1; j++) {
64             if (destino[j] > destino[j + 1]) {
65                 int temporario = destino[j];
66                 destino[j] = destino[j + 1];
67                 destino[j + 1] = temporario;
68             }
69         }
70     }
71 }
72
73 void imprimirVetor(const char *rotulo, const int valores[], uint8_t tamanho) {
74     Serial.print(rotulo);
75     for (uint8_t i = 0; i < tamanho; i++) {
76         Serial.print(valores[i]);
77         if (i < tamanho - 1) {
78             Serial.print(' ');
79         }
80     }
81     Serial.println();
82 }

```

Continua - bloco 2 de 3

```

83
84 void aplicarHisterese(int mediana) {
85     EstadoVegetacao estadoAnterior = estadoAtual;
86
87     if (mediana >= 16) {
88         estadoAtual = ALERTA;
89     } else if (mediana <= 14) {
90         estadoAtual = NORMAL;
91     }
92
93     Serial.printf("Estado anterior: %s\n", nomeDoEstado(estadoAnterior));
94     if (mediana > 14 && mediana < 16) {
95         Serial.println("Faixa de histerese: estado anterior mantido.");
96     } else if (estadoAnterior == estadoAtual) {
97         Serial.println("Limiar confirmado: estado permanece inalterado.");
98     } else {
99         Serial.println("Limiar cruzado: estado alterado.");
100    }
101    Serial.printf("Estado atual: %s\n", nomeDoEstado(estadoAtual));
102
103    atualizarLedDoEstado();
104    Serial.println(estadoAtual == ALERTA
105                  ? "LED VERMELHO: ESTADO ALERTA"
106                  : "LED VERDE: ESTADO NORMAL");
107 }
108
109 void iniciarSessao(unsigned long inicioProgramado) {
110     indiceLeitura = 0;
111     sessaoAtiva = true;
112     proximaLeituraEm = inicioProgramado;
113
114     Serial.println();
115     Serial.println("-----");
116     Serial.printf("INICIO DA SESSAO %u - FW %s\n", sessoesConcluidas + 1, VERSAO_ATUAL);
117     Serial.println("-----");
118 }
119
120 void concluirSessao() {
121     int leiturasOrdenadas[TOTAL_LEITURAS];
122     ordenarCopia(leituras, leiturasOrdenadas, TOTAL_LEITURAS);
123
124     long soma = 0;
125     for (uint8_t i = 0; i < TOTAL_LEITURAS; i++) {
126         soma += leituras[i];
127     }
128
129     float media = soma / static_cast<float>(TOTAL_LEITURAS);
130     int mediana = leiturasOrdenadas[TOTAL_LEITURAS / 2];
131
132     imprimirVetor("Leituras na ordem original: ", leituras, TOTAL_LEITURAS);
133     imprimirVetor("Leituras em ordem crescente: ", leiturasOrdenadas, TOTAL_LEITURAS);
134     Serial.printf("Media da sessao: %.1f cm\n", media);
135     Serial.printf("Mediana da sessao: %d cm\n", mediana);
136     aplicarHisterese(mediana);
137
138     sessoesConcluidas++;
139     sessaoAtiva = false;
140     Serial.printf("Sessao %u concluida.\n", sessoesConcluidas);
141     Serial.println("Proxima sessao no marco de 48 segundos.");
142 }
143
144 void processarLeituras(unsigned long agora) {
145     if (!sessaoAtiva || indiceLeitura >= TOTAL_LEITURAS ||
146         !instanteAtingido(agora, proximaLeituraEm)) {
147         return;
148     }
149
150     leituras[indiceLeitura] = gerarAltura();
151     Serial.printf("Leitura %u: %d cm\n", indiceLeitura + 1, leituras[indiceLeitura]);
152     indiceLeitura++;
153     proximaLeituraEm += INTERVALO_LEITURA_MS;
154
155     if (indiceLeitura == TOTAL_LEITURAS) {
156         concluirSessao();
157     }
158 }
159
160 void setup() {
161     Serial.begin(115200);
162     pinMode(PINO_LED_VERMELHO, OUTPUT);
163     pinMode(PINO_LED_VERDE, OUTPUT);
164     pinMode(PINO_LED_AZUL, OUTPUT);

```

Continua - bloco 3 de 3

```
165     estadoAtual = NORMAL;
166     atualizarLedDoEstado();
167
168     Serial.println();
169     Serial.println("=====");
170     Serial.println("MONITORAMENTO DE VEGETACAO - FW 2.0");
171     Serial.println("MEDIA + ORDENACAO + MEDIANA + HISTERESE");
172     Serial.println("=====");
173     Serial.println("Firmware atualizado por OTA e reiniciado com sucesso.");
174
175     unsigned long agora = millis();
176     iniciarSessao(agora);
177     proximaSessaoEm = agora + INTERVALO_SESSAO_MS;
178 }
179
180 void loop() {
181     unsigned long agora = millis();
182     processarLeituras(agora);
183
184     if (!sessaoAtiva && instanteAtingido(agora, proximaSessaoEm)) {
185         unsigned long inicioProgramado = proximaSessaoEm;
186         proximaSessaoEm += INTERVALO_SESSAO_MS;
187         iniciarSessao(inicioProgramado);
188     }
189
190     delay(10);
191 }
```

Apendice C - Manifesto version.json

```

1 {
2   "version": "2.0",
3   "url": "https://raw.githubusercontent.com/vitorlimeira22/motiva-ota-esp32-cp2/main/firmware_v2.bin"
4 }

```

Apendice D - Testes automatizados

```

1 from pathlib import Path
2 import json
3 import re
4 import unittest
5
6
7 ROOT = Path(__file__).resolve().parent
8
9
10 def gerar_sequencia(seed: int, quantidade: int) -> list[int]:
11     estado = seed
12     valores = []
13     for _ in range(quantidade):
14         estado = (1664525 * estado + 1013904223) & 0xFFFFFFFF
15         valores.append(10 + estado % 11)
16     return valores
17
18
19 def aplicar_histerese(media: int, estado_anterior: str) -> str:
20     if media >= 16:
21         return "ALERTA"
22     if media <= 14:
23         return "NORMAL"
24     return estado_anterior
25
26
27 class TesteFirmware(unittest.TestCase):
28     def test_gerador_fica_entre_10_e_20(self):
29         valores = gerar_sequencia(87, 1000)
30         self.assertTrue(all(10 <= valor <= 20 for valor in valores))
31
32     def test_primeiras_sessoes_cobrem_histerese(self):
33         valores = gerar_sequencia(87, 15)
34         medias = [sorted(valores[i:i + 5])[2] for i in range(0, 15, 5)]
35         self.assertEqual([18, 15, 14], medias)
36
37         estado = "NORMAL"
38         estados = []
39         for media in medias:
40             estado = aplicar_histerese(media, estado)
41             estados.append(estado)
42         self.assertEqual(["ALERTA", "ALERTA", "NORMAL"], estados)
43
44     def test_temporizacao_das_sessoes(self):
45         inicios = [0 + indice * 48000 for indice in range(4)]
46         leituras = [[inicio + passo * 2000 for passo in range(5)] for inicio in inicios]
47         self.assertEqual([0, 2000, 4000, 6000, 8000], leituras[0])
48         self.assertEqual(48000, leituras[1][0])
49         self.assertEqual(48000, inicios[1] - inicios[0])
50
51     def test_manifesto_tem_campos_obrigatorios(self):
52         manifesto = json.loads((ROOT / "version.json").read_text(encoding="utf-8"))
53         self.assertEqual("2.0", manifesto["version"])
54         self.assertRegex(manifesto["url"], r"^https://")
55
56     def test_codigo_contem_tratamentos_minimos(self):
57         codigo = (ROOT / "firmware_v1.ino").read_text(encoding="utf-8")
58         mensagens = [
59             "Nao foi possivel conectar ao Wi-Fi",
60             "Manifesto indisponivel",
61             "A versao instalada ja e a mais recente",
62             "Download retornou HTTP",
63             "Falha ao finalizar a atualizacao",
64         ]
65         for mensagem in mensagens:
66             self.assertIn(mensagem, codigo)
67
68     def test_codigo_v2_possui_evolucao_real(self):
69         codigo = (ROOT / "firmware_v2.ino").read_text(encoding="utf-8")
70         for elemento in ["ordenarCopia", "Mediana da sessao", "aplicarHisterese", "LED VERMELHO"]:
71             self.assertIn(elemento, codigo)
72         self.assertIsNone(re.search(r"delay\s*\(\s*48000\s*\)", codigo))
73
74
75 if __name__ == "__main__":
76     unittest.main(verbosity=2)

```